

NumPy / pandas Reference for searcher.py (Assignment 1)

COMP 3625. Tools you could use instead of hand-written code. "Now" = fits what the file already does. "Soon" = likely to matter as the project grows.

Examples assume: `import numpy as np, pandas as pd, numbers; rng = np.random.default_rng(42); curr = np.array([0.5, 0.5]); scores = np.array([0.2, 0.9, 0.4, 0.7]); pts = np.array([[0.5, 0.5], [0.6, 0.5], [0.5, 0.5]]); s = pd.Series({(0.5, 0.5): 0.3, (0.6, 0.5): 0.4}); df = pd.DataFrame([(0.5, 0.5, 0.31), (0.6, 0.5, 0.42), (0.5, 0.5, 0.31)], columns=['k0', 'k1', 'score'])`. The green line under each example is its result.

Core NumPy data structures

ndarray is the main structure: a fixed-size grid of values that all share one type, stored in one contiguous block of memory. That is why it is fast, and why it differs from a Python list.

Attribute	Meaning
shape	Tuple of sizes per dimension. A 10-knob point is (10,). A batch of 20 points is (20, 10).
ndim	Number of dimensions (len(shape)).
dtype	The one element type, e.g. float64 or int64. Mixing types forces everything to the widest one.
size	Total element count.
axis	The dimension number that reductions and sorts operate along.

Axis. In a 2-D array, `axis=0` runs down the rows, so `a.max(axis=0)` gives one result per column. `axis=1` runs across the columns, so `a.max(axis=1)` gives one result per row. The axis you name is the one that disappears.

Vectorisation and broadcasting. Operations apply to every element at once, with no Python loop. If shapes differ, NumPy lines them up from the right and stretches any dimension of size 1. A (10,) vector plus a (20, 10) matrix adds the vector to every row. A (20, 1) column plus a (10,) vector gives a (20, 10) grid. Shapes that cannot be aligned raise an error.

Views vs copies. Slicing (`a[2:5]`) returns a view onto the same memory, so editing it edits the original. Boolean-mask and integer-list indexing returns a copy. `.copy()` forces independence.

Indexing forms. Slicing: `a[1:4]`, `a[:, 0]` (first column). Boolean mask: `a[a > 0.5]` keeps elements where the test is true. Integer array: `a[[3, 1, 1]]` picks specific positions, repeats allowed.

Mutability and hashing. Arrays are mutable, so they are unhashable: they cannot be dict keys or set members. Convert with `.tolist()` and then `tuple(...)` when you need one as a key.

NumPy scalars. Reducing an array gives an `np.float64` or `np.int64`, not a Python float or int. `np.float64` behaves and hashes like a float. `np.int64` is not an int subclass, so `isinstance(x, int)` fails on it.

Special values. `np.nan` means "not a number" and never equals itself. `np.inf` is infinity.

np.random.Generator is the modern random source, created with `default_rng(seed)`. It holds random state, and all draws are methods on it. The same seed reproduces the same run.

Other structures. Structured arrays (rows with named, mixed-type fields); masked arrays, `np.ma` (some entries flagged invalid); `np.matrix` (legacy, discouraged: prefer 2-D ndarray).

pandas counterparts. Series = 1-D labelled array (an ndarray plus an index). DataFrame = 2-D table of Series sharing one index, with a dtype per column. Index / MultiIndex = the labels; a MultiIndex has several label levels, and tuple keys create one automatically.

1. Creating and converting arrays

Name	Inputs (type: meaning)	Returns (type)	Example of use	Notes	When
<code>np.array(x, dtype=float)</code>	x (list, tuple or array_like): the values dtype (type or str, optional): element type, e.g. float	ndarray. Shape follows the nesting of x.	<code>np.array([0.5, 0.2], dtype=float)</code> > array([0.5, 0.2])	Enables vector math with no per-knob loop.	Now
<code>np.ones(n) / np.full(n, v) / np.zeros(n)</code>	n (int or tuple of ints): shape v (scalar, full only): fill value dtype (optional)	ndarray of shape n, filled with 1, v or 0.	<code>np.full((2, 3), np.nan)</code> > 2x3 array, every entry nan	<code>np.full(n, step)</code> is a step vector for moving every knob at once.	Now
<code>np.eye(N, dtype=float)</code>	N (int): number of rows and columns dtype (optional, default float64)	ndarray, shape (N, N), 1.0 on the diagonal.	<code>np.eye(2) * 0.1</code> > array([[0.1, 0.], [0., 0.1]])	Each row is a unit vector, i.e. "one knob only". Multiply by a scalar to get a single-knob move.	Now
<code>np.linspace(start, stop, num) / np.arange(start, stop, step)</code>	start, stop (float): range ends num (int, default 50): point count (linspace) endpoint (bool, default True): include stop step (int or float): spacing (arange)	1-D ndarray of float64. Length is num for linspace, ceil((stop-start)/step) for arange.	<code>np.linspace(0, 1, 5)</code> > array([0., 0.25, 0.5, 0.75, 1.])	linspace includes the endpoint by default. arange has float-step accumulation issues.	Soon
<code>ndarray.tolist()</code>	none (method on an ndarray)	list of Python scalars (nested lists if ndim > 1).	<code>np.array([0.5, 0.3]).tolist()</code> > [0.5, 0.3]	<code>tuple(arr)</code> leaves np.float64 elements. Arrays are unhashable, so convert before using as a key.	Now
<code>np.copy(a) / a.copy()</code>	a (array_like): the array to copy	New ndarray, same shape and dtype.	<code>a = np.array([1., 2.])</code> <code>b = a.copy()</code> <code>b[0] = 9</code> <code>a</code> > array([1., 2.]) (a unchanged)	Slices are views, so copy before editing.	Now
<code>pd.DataFrame(rows, columns=[...])</code>	rows (list of tuples or dicts, dict, or ndarray): the data columns (list of str): column names	DataFrame. One row per input record.	<code>pd.DataFrame([(0.5, 0.5, 0.31), (0.6, 0.5, 0.42)], columns=['k0', 'k1', 'score'])</code> > 2 rows, columns k0, k1, score	One row per evaluation is the natural shape for a score log.	Now
<code>pd.Series(dict)</code>	data (dict, list or ndarray): the values index (list-like, optional): labels name (str, optional)	Series. Dict keys become the index.	<code>pd.Series({(0.5, 0.5): 0.3, (0.6, 0.5): 0.4})</code> > Series of 2 values, MultiIndex of the tuples	Tuple keys become a MultiIndex.	Now
<code>DataFrame.to_numpy()</code>	dtype (optional) copy (bool, default False)	ndarray (2-D for a DataFrame).	<code>df[['k0', 'k1']].to_numpy()</code> > array([[0.5, 0.5], [0.6, 0.5], [0.5, 0.5]])	Use it to hand a row to code that expects arrays.	Soon

2. Element-wise math and bounding

Name	Inputs (type: meaning)	Returns (type)	Example of use	Notes	When
Broadcasting (arr + scalar, arr + arr2d)	two operands: arrays, or an array and a scalar (int or float)	ndarray with the broadcast shape.	<code>curr + np.eye(2) * 0.1</code> > <code>array([[0.6, 0.5], [0.5, 0.6]])</code>	A (n,) vector plus an (m, n) matrix adds the vector to every row.	Now
<code>np.clip(a, a_min, a_max)</code>	a (array_like or scalar): values a_min, a_max (scalar or array_like; None skips that side): limits	ndarray, same shape as a. np.float64 if a is a scalar.	<code>np.clip(np.array([-0.1, 0.5, 1.2]), 0, 1)</code> > <code>array([0., 0.5, 1.])</code>	NumPy 2.1+ also accepts min=/max= keywords.	Now
<code>np.minimum(a, b)</code> / <code>np.maximum(a, b)</code>	a, b (array_like or scalars): broadcast against each other	ndarray of element-wise min or max (np scalar for scalar inputs).	<code>np.minimum(np.array([0.6, 1.2]), 1.0)</code> > <code>array([0.6, 1.])</code>	Different from np.min/np.max, which reduce one array to one value.	Now
<code>np.round(a, decimals=0)</code>	a (array_like or scalar): values decimals (int, default 0; negative allowed): places to keep	ndarray, same shape. np.float64 for a scalar.	<code>np.round(np.array([0.30000000000000004, 0.7]), 10)</code> > <code>array([0.3, 0.7])</code>	Round-half-to-even.	Now
<code>np.round(x / step) * step</code> (idiom)	x (float or ndarray): value(s) step (float): grid spacing	Same type as x. Values snapped to the nearest multiple of step.	<code>np.round(0.34 / 0.1) * 0.1</code> > <code>about 0.3 (3 * 0.1)</code>	Keeps points on a consistent grid.	Soon
<code>np.exp(x)</code>	x (array_like or scalar)	ndarray (np.float64 for a scalar) of e**x.	<code>np.exp(-0.5)</code> > <code>np.float64(0.6065...)</code>	Needed for simulated annealing's accept probability.	Soon
<code>DataFrame.clip(lower, upper) / .round(decimals)</code>	lower, upper (float or None): limits decimals (int): places to keep	DataFrame, same shape.	<code>df[['k0', 'k1']].clip(lower=0.55, upper=1)</code> > <code>every value below 0.55 becomes 0.55</code>	Handy if points are stored as rows.	Now

3. Comparing values and points

Name	Inputs (type: meaning)	Returns (type)	Example of use	Notes	When
<code>np.isclose(a, b, rtol=1e-5, atol=1e-8)</code>	a, b (array_like or scalars): values to compare rtol (float): relative tolerance atol (float): absolute tolerance	ndarray of bool (np.bool_ for scalars).	<code>np.isclose(0.1 + 0.2, 0.3)</code> > <code>np.True_</code>	Set atol yourself if your values are small.	Now
<code>np.allclose(a, b, ...)</code>	same as isclose, plus equal_nan (bool, default False)	Python bool: True only if every element is close.	<code>np.allclose([0.5, 0.5], [0.5, 0.5000001])</code> > <code>True</code>	Use it for "are these the same point?"	Now
<code>np.array_equal(a1, a2)</code>	a1, a2 (array_like)	Python bool. True only if shapes and all elements match exactly.	<code>np.array_equal([0.5, 0.5], [0.5, 0.5])</code> > <code>True</code>	Float noise still breaks it.	Now
<code>np.linalg.norm(x, ord=None, axis=None)</code>	x (array_like): vector or matrix ord (None, int, float or np.inf): norm type; None = 2-norm axis (int or None): axis to reduce	np.float64, or ndarray of floats when axis is given.	<code>np.linalg.norm(np.array([0.6, 0.5]) - curr)</code> > <code>about 0.1</code>	Pass a difference of two points to get their distance. axis=1 gives a norm per row.	Soon
<code>np.testing.assert_allclose(actual, desired, rtol, atol)</code>	actual, desired (array_like) rtol, atol (float): tolerances	None. Raises AssertionError on mismatch.	<code>np.testing.assert_allclose([0.3], [0.1 + 0.2])</code> > <code>None (passes silently)</code>	Prints a readable mismatch report on failure.	Soon

4. Finding extremes and selecting

Name	Inputs (type: meaning)	Returns (type)	Example of use	Notes	When
<code>np.max(a, axis) / np.min(a, axis)</code>	a (array_like): values axis (int or None): axis to reduce	np scalar (axis=None) or ndarray.	<code>np.max(scores)</code> > <code>np.float64(0.9)</code>	Pair with <code>argmax</code> when you need both value and position.	Now
<code>np.argmax(a, axis=None) / np.argmin</code>	a (array_like): values axis (int or None): axis to search	np.int64 index into the flattened array (axis=None), or ndarray of ints.	<code>np.argmax(scores)</code> > <code>np.int64(1)</code>	First occurrence wins on ties, which is a deterministic tie-break.	Now
<code>np.where(cond, x, y)</code>	cond (array of bool): the test x, y (array_like, optional): values where true / false	ndarray (3 args). With only cond: tuple of index arrays.	<code>np.where(scores > 0.5)</code> > <code>array([1, 3],)</code>	See also <code>np.flatnonzero</code> for 1-D index lists.	Now
<code>np.any(a, axis) / np.all(a, axis)</code>	a (array_like): usually a bool array axis (int or None)	np.bool_, or ndarray of bool.	<code>np.all(pts == [0.5, 0.5], axis=1)</code> > <code>array([True, False, True])</code>	axis gives a per-row or per-column answer.	Now
<code>np.argpartition(a, kth)</code>	a (array_like): values kth (int or sequence of int): position(s) to guarantee axis (int, default -1)	ndarray of int indices. Only position kth is guaranteed in sorted place.	<code>np.sort(np.argpartition(scores, -2)[-2:])</code> > <code>array([1, 3])</code> (positions of the 2 largest)	Cheaper than <code>argsort</code> when you only need the top few.	Soon
<code>Series.idxmax() / .idxmin()</code>	skipna (bool, default True): ignore NaN	One index label (a tuple if the index is a MultiIndex).	<code>s.idxmax()</code> > <code>(0.6, 0.5)</code>	First label on ties.	Now
<code>Series.max() / .nlargest(n)</code>	n (int, default 5): how many to return keep ("first", "last" or "all"): tie handling	max: scalar. nlargest: Series of n rows, largest first.	<code>s.nlargest(1)</code> > <code>one-row Series: (0.6, 0.5) -> 0.4</code>	Use <code>ascending=False</code> on sorts for descending order.	Now

5. Sorting and ordering

Name	Inputs (type: meaning)	Returns (type)	Example of use	Notes	When
<code>np.sort(a, axis=-1) / np.argsort(a)</code>	a (array_like): values axis (int, default -1): axis to sort along kind (str, optional): algorithm, e.g. "stable"	sort: ndarray, sorted copy. argsort: ndarray of int indices.	<code>np.argsort([0.4, 0.1, 0.9])</code> > <code>array([1, 0, 2])</code>	<code>np.sort(a, axis=0)</code> sorts each column independently, which scrambles rows.	Now
<code>np.lexsort(keys)</code>	keys (sequence of 1-D arrays, or 2-D array): sort keys axis (int, default -1)	1-D ndarray of int indices, length N.	<code>pts[np.lexsort(pts.T[:-1])]</code> > <code>array([[0.5, 0.5], [0.5, 0.5], [0.6, 0.5]])</code>	The last key is the primary key. Pass columns as <code>arr.T</code> .	Now
<code>Series.sort_values(ascending, kind)</code>	ascending (bool, default True) kind (str): use "stable" to keep tie order	Series, sorted copy.	<code>s.sort_values(ascending=False, kind='stable')</code> > <code>Series ordered 0.4 first, then 0.3</code>	Stable sorting keeps "first scored" order among equals.	Now

6. Sets, duplicates and membership

Name	Inputs (type: meaning)	Returns (type)	Example of use	Notes	When
<code>np.unique(a, axis=0, return_index=True)</code>	<code>ar</code> (array_like): values <code>axis</code> (int or None): axis to compare along; 0 = whole rows <code>return_index</code> (bool): also return first-occurrence positions	ndarray of unique values or rows. With <code>return_index</code> : tuple (unique, indices).	<code>np.unique(pts, axis=0)</code> > <code>array([[0.5, 0.5], [0.6, 0.5]])</code>	Output is sorted lexicographically.	Now
<code>np.isin(element, test_elements)</code>	<code>element</code> (array_like): values to test <code>test_elements</code> (array_like): the set to test against	ndarray of bool, same shape as <code>element</code> .	<code>np.isin([1, 5], [1, 2, 3])</code> > <code>array([True, False])</code>	For whole rows, compare with <code>(a[:, None] == b).all(axis=2).any(axis=1)</code> .	Now
<code>np.setdiff1d(ar1, ar2)</code>	<code>ar1, ar2</code> (1-D array_like)	Sorted 1-D ndarray of unique values in <code>ar1</code> not in <code>ar2</code> .	<code>np.setdiff1d([1, 2, 3, 4], [2, 4])</code> > <code>array([1, 3])</code>	1-D only, and it sorts the result.	Now
<code>DataFrame.drop_duplicates(subset=None, keep="first")</code>	<code>subset</code> (label, list of labels, or None): columns to compare <code>keep</code> ("first", "last" or False): which duplicate survives	DataFrame with duplicate rows removed.	<code>df.drop_duplicates(subset=['k0', 'k1'], keep='first')</code> > DataFrame with 2 rows (the repeat, row 2, is dropped)	<code>keep="first"</code> matches "first scored wins".	Now
<code>DataFrame.duplicated(subset, keep)</code>	same as <code>drop_duplicates</code>	Series of bool: True marks a repeated row.	<code>df.duplicated(subset=['k0', 'k1'])</code> > <code>Series: False, False, True</code>	Use it as a mask to inspect repeats.	Now
<code>pd.Index.difference(other, sort=None)</code>	<code>other</code> (Index or array_like): labels to remove <code>sort</code> (None or False): False keeps order	Index of labels in this index but not in <code>other</code> .	<code>pd.Index([1, 2, 3]).difference(pd.Index([2]))</code> > <code>Index([1, 3])</code>	Sorts by default. Pass <code>sort=False</code> to keep order.	Now
<code>pd.MultiIndex.from_tuples(tuples, names)</code>	<code>tuples</code> (list of tuples): one per row <code>names</code> (list of str, optional): level names	MultiIndex.	<code>pd.MultiIndex.from_tuples([(0.5, 0.5), (0.6, 0.5)], names=['k0', 'k1'])</code> > MultiIndex with levels <code>k0, k1</code>	One level per knob, so you can filter by knob value.	Soon

7. Stacking and building move patterns

Name	Inputs (type: meaning)	Returns (type)	Example of use	Notes	When
<code>np.vstack(tup) / np.concatenate(tup, axis=0)</code>	<code>tup</code> (sequence of arrays): arrays to join <code>axis</code> (int, default 0): axis to join along	ndarray. Rows of each input, one after another (<code>axis=0</code>).	<code>np.vstack([(0.6, 0.5), [0.4, 0.5]])</code> > <code>array([[0.6, 0.5], [0.4, 0.5]])</code>	Shapes must agree on the other axes.	Now
<code>np.tri(N, M=None, k=0) / np.tril(m, k) / np.triu(m, k)</code>	<code>N</code> (int): rows <code>M</code> (int, optional): columns <code>k</code> (int): diagonal offset <code>m</code> (array_like): array to mask (<code>tril/triu</code>)	<code>tri</code> : ndarray (N, M) of 1.0/0.0. <code>tril/triu</code> : copy of <code>m</code> with entries outside the triangle set to 0.	<code>np.tri(3)</code> > <code>array([[1., 0., 0.], [1., 1., 0.], [1., 1., 1.]])</code>	Row <code>i</code> of <code>tri</code> is 1 up to column <code>i</code> : a "first <code>i</code> knobs" mask. Multiply by a step to move a prefix of knobs together.	Soon
<code>np.cumsum(a, axis) / np.diff(a, n=1, axis)</code>	<code>a</code> (array_like): values <code>axis</code> (int, optional) <code>n</code> (int, default 1): times to difference	<code>cumsum</code> : ndarray, same shape. <code>diff</code> : ndarray, <code>n</code> shorter along axis.	<code>np.diff([1., 4., 9.])</code> > <code>array([3., 5.])</code>	Useful for cumulative-step patterns.	Soon

8. Special values and type checking

Name	Inputs (type: meaning)	Returns (type)	Example of use	Notes	When
<code>np.inf, -np.inf</code>	none (constants)	float (positive or negative infinity).	<code>0.3 > -np.inf</code> > True	Cleaner than <code>float("inf")</code> .	Now
<code>np.nan, np.isnan(x)</code>	x (array_like or scalar): values to test	nan: float. isnan: ndarray of bool (np.bool_ for a scalar).	<code>np.isnan(np.nan)</code> > np.True_	<code>nan == nan</code> is False, so use <code>isnan</code> .	Now
<code>isinstance(x, (int, np.integer)) / np.floating</code>	x (any object) class or tuple of classes	Python bool.	<code>isinstance(np.int64(3), (int, np.integer))</code> > True	<code>np.bool_</code> is not an <code>np.integer</code> , but Python <code>bool</code> is an <code>int</code> .	Now
<code>numbers.Real (stdlib)</code>	x (any object), used with <code>isinstance</code>	Python bool.	<code>isinstance(0.5, numbers.Real)</code> > True	Also matches <code>bool</code> , so exclude it explicitly.	Now
<code>np.issubdtype(arg1, arg2)</code>	arg1 (dtype or type): e.g. <code>arr.dtype</code> arg2 (dtype or abstract type): e.g. <code>np.floating</code>	Python bool.	<code>np.issubdtype(np.array([0.5]).dtype, np.floating)</code> > True	Use for validating array inputs.	Soon

9. Randomness

Name	Inputs (type: meaning)	Returns (type)	Example of use	Notes	When
<code>np.random.default_rng(seed)</code>	seed (int, array_like of int, or None): None = fresh entropy	Generator object holding the random state.	<code>np.random.default_rng(42)</code> > Generator object (same seed = same draws)	Prefer it over <code>np.random.seed</code> . Same seed gives the same run.	Soon
<code>rng.uniform(low, high, size) / rng.random(size)</code>	low, high (float): range (uniform) size (int, tuple or None): output shape	float (size=None) or ndarray of float64. random draws from [0, 1).	<code>np.random.default_rng(42).uniform(0, 1, size=2)</code> > array([0.77395605, 0.43887844])	Use for random starts and restarts.	Soon
<code>rng.choice(a, size, replace, p)</code>	a (int or 1-D array_like): pool size (int, tuple or None) replace (bool, default True) p (array_like of float, optional): probabilities	One element (size=None) or ndarray of elements.	<code>rng.choice(4, size=2, replace=False)</code> > e.g. array([2, 0]) (random, no repeats)	Use for stochastic neighbour selection.	Soon
<code>rng.permutation(x) / rng.shuffle(x)</code>	x (int or array_like): items to reorder	permutation: new shuffled ndarray. shuffle: None (reorders x in place).	<code>x = np.arange(4)</code> <code>res = rng.shuffle(x)</code> <code>res</code> > None (x itself is reordered)	Randomised tie-breaking or neighbour order.	Soon
<code>rng.normal(loc, scale, size)</code>	loc (float): mean scale (float): standard deviation size (int, tuple or None)	float or ndarray of float64.	<code>rng.normal(0, 0.05, size=2)</code> > e.g. array([0.01, -0.04]) (random, centred on 0)	Use for random-perturbation moves.	Soon

10. Surfaces, gradients and plotting

Name	Inputs (type: meaning)	Returns (type)	Example of use	Notes	When
<code>np.meshgrid(*xi, indexing="xy")</code>	<code>xi</code> (1-D arrays): one per axis indexing ("xy" or "ij")	Tuple of ndarrays, one per axis. For 2 inputs, each has shape $(\text{len}(y), \text{len}(x))$.	<code>np.meshgrid(np.linspace(0, 1, 3), np.linspace(0, 1, 3))[0].shape > (3, 3)</code> (one such array per axis)	Score every grid point to see the 2-D surface.	Soon
<code>np.gradient(f, *varargs)</code>	<code>f</code> (array_like): sampled values <code>varargs</code> (scalars or coordinate arrays): spacing per axis	ndarray (1-D <code>f</code>) or list of ndarrays, one per axis.	<code>np.gradient(np.array([1., 4., 9.])) > array([3., 4., 5.])</code>	Groundwork for gradient-style moves and surface plots.	Soon
<code>Series.cummax()</code>	<code>skipna</code> (bool, default True)	Series, same length: the running maximum.	<code>pd.Series([0.2, 0.5, 0.4]).cummax().tolist() > [0.2, 0.5, 0.5]</code>	Gives a best-so-far-per-evaluation curve.	Soon
<code>Series.plot()</code> / <code>DataFrame.plot()</code>	<code>kind</code> (str, default "line") <code>x, y</code> (label): columns (DataFrame)	matplotlib Axes.	<code>type(pd.Series([0.2, 0.5, 0.4]).plot(kind='line')).__name__ > Axes</code> (a matplotlib Axes)	Good for score-vs-evaluation graphs.	Soon
<code>np.vectorize(pyfunc)</code>	<code>pyfunc</code> (callable): a scalar function <code>otypes</code> (str or list, optional): output types	A callable that accepts arrays and returns an ndarray.	<code>np.vectorize(lambda v: min(1.0, max(0.0, v)))([-0.1, 0.5, 1.2]) > array([0., 0.5, 1.])</code>	A convenience loop, not a speedup.	Soon

Check your understanding

- For a (20, 10) array of 20 candidate points, what shape does `argmax(axis=1)` return, and what does each entry mean?
- Why does `a[2:5]` behave differently from `a[[2, 3, 4]]` when you edit the result?